
EEE5776. Applied Machine Learning



Module 1. Introduction to Libraries and Frameworks

A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps. The lines are of varying lengths and are arranged in a way that creates a sense of depth and movement.

Conda Environment

Conda Environment and Jupyter Notebooks

1. Install the Anaconda Package (it includes Python 3.12)

- If you are installing Anaconda for the first time, this will create a base environment with all the Anaconda libraries installed and ready to run.

2. Create a new environment and install all packages/libraries.

- `conda create --name aml`
- `conda activate aml`
- `conda install anaconda`

3. Customize your environment by installing Jupyter Notebook extensions.

- `conda install -c conda-forge jupyter_contrib_nbextensions`

4. Launching Jupyter Notebooks

- `conda activate aml`
- `jupyter notebook`



Modules and Libraries

Our Ecosystem

- Many of the tools we will use in the class are not directly part of Python.
- Instead, they are libraries or modules that provide particular functionality. These include:



matplotlib



NumPy (Numerical Python)

- **Purpose:** Efficient numerical computation.
- **Key Feature:** Provides ndarrays (N-dimensional arrays) for fast mathematical operations.
- **Functionality:**
 - Linear algebra operations (dot products, eigenvalues, matrix decompositions)
 - Element-wise operations and broadcasting
 - Random number generation
 - Integration with C/C++ for performance
- **Similar to MATLAB:** Core numerical and matrix operations.
- **Use Case:** Fast mathematical computation on large datasets or numerical simulations.

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.dot(a, b)) # Output: 32 (dot product)
```



SciPy (Scientific Python)

- **Purpose:** Advanced scientific and technical computing built on top of NumPy.
- **Key Feature:** Provides specialized modules for scientific computations.
- **Functionality:**
 - **Optimization** (scipy.optimize): Solving linear and nonlinear problems.
 - **Integration** (scipy.integrate): Numerical integration (e.g., solving differential equations).
 - **Linear Algebra** (scipy.linalg): Advanced matrix operations beyond NumPy.
 - **Signal Processing** (scipy.signal): Filtering and signal analysis.
 - **Statistics** (scipy.stats): Probability distributions, statistical tests.
- **Use Case:** For researchers, engineers, and scientists needing mathematical routines.

```
from scipy.optimize import minimize

# Define a simple quadratic function
def func(x):
    return (x - 3)**2

result = minimize(func, x0=0) # Minimize starting from x=0
print(result.x) # Output: [3.] (minimum at x = 3)
```

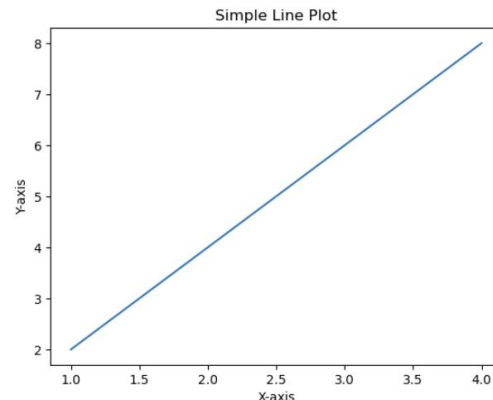


Matplotlib

- **Purpose:** Data visualization.
- **Key Feature:** Plotting 2D and 3D graphs (similar to MATLAB's plotting capabilities).
- **Functionality:**
 - Line plots, bar charts, scatter plots, histograms, heatmaps, etc.
 - Customizable plots (titles, labels, legends, colors).
 - Support for interactive and static visualizations.
- **Similar to MATLAB:** Plotting functions and figure customization.
- **Use Case:** Visualizing data for exploration, analysis, and presentation.

matplotlib

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]
plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Simple Line Plot')
plt.show()
```



- **Purpose:** Data manipulation and analysis.
- **Key Feature:** Provides **DataFrame** and **Series** for structured data handling.
- **Functionality:**
 - Reading/writing data (CSV, Excel, SQL, JSON)
 - Data cleaning and preprocessing (handling missing data, filtering, grouping)
 - Statistical analysis and data aggregation
 - Time series analysis
- **Similar to MATLAB:** Data table management but more powerful for handling diverse data formats.
- **Use Case:** Organizing, cleaning, transforming, and analyzing tabular data.

```
import pandas as pd
data = {'Name': ['Alice', 'Bob'], 'Age': [25, 30]}
df = pd.DataFrame(data)
print(df)
```



scikit-learn (Machine Learning in Python)

- **Purpose:** Easy-to-use tools for machine learning and statistical modeling.
- **Key Feature:** High-level API for building, training, and evaluating models.
- **Functionality:**
 - **Supervised Learning:** Classification, regression (e.g., Decision Trees, SVM, Linear Regression).
 - **Unsupervised Learning:** Clustering, dimensionality reduction (e.g., K-Means, PCA).
 - **Model Selection:** Cross-validation, hyperparameter tuning (GridSearchCV).
 - **Preprocessing:** Data scaling, encoding, and feature selection.
- **Use Case:** For building machine learning models quickly without diving deep into customization.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load Iris dataset
X, y = load_iris(return_X_y=True)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```



Our Ecosystem

- When you install the Anaconda distribution, all of these libraries will be installed in your machine!
 - If all these are included, why to create a dedicated environment?
 - **Dependency Isolation.** Different projects may require different versions of the same package. A dedicated environment ensures that dependencies don't conflict. For example: It depends on your use case, but generally, it's better to avoid installing PyTorch and TensorFlow in the same environment due to potential dependency conflicts and resource management issues



Our Ecosystem

**Let's play a little bit in a jupyter notebook
that will be available in our repository in
Github!**